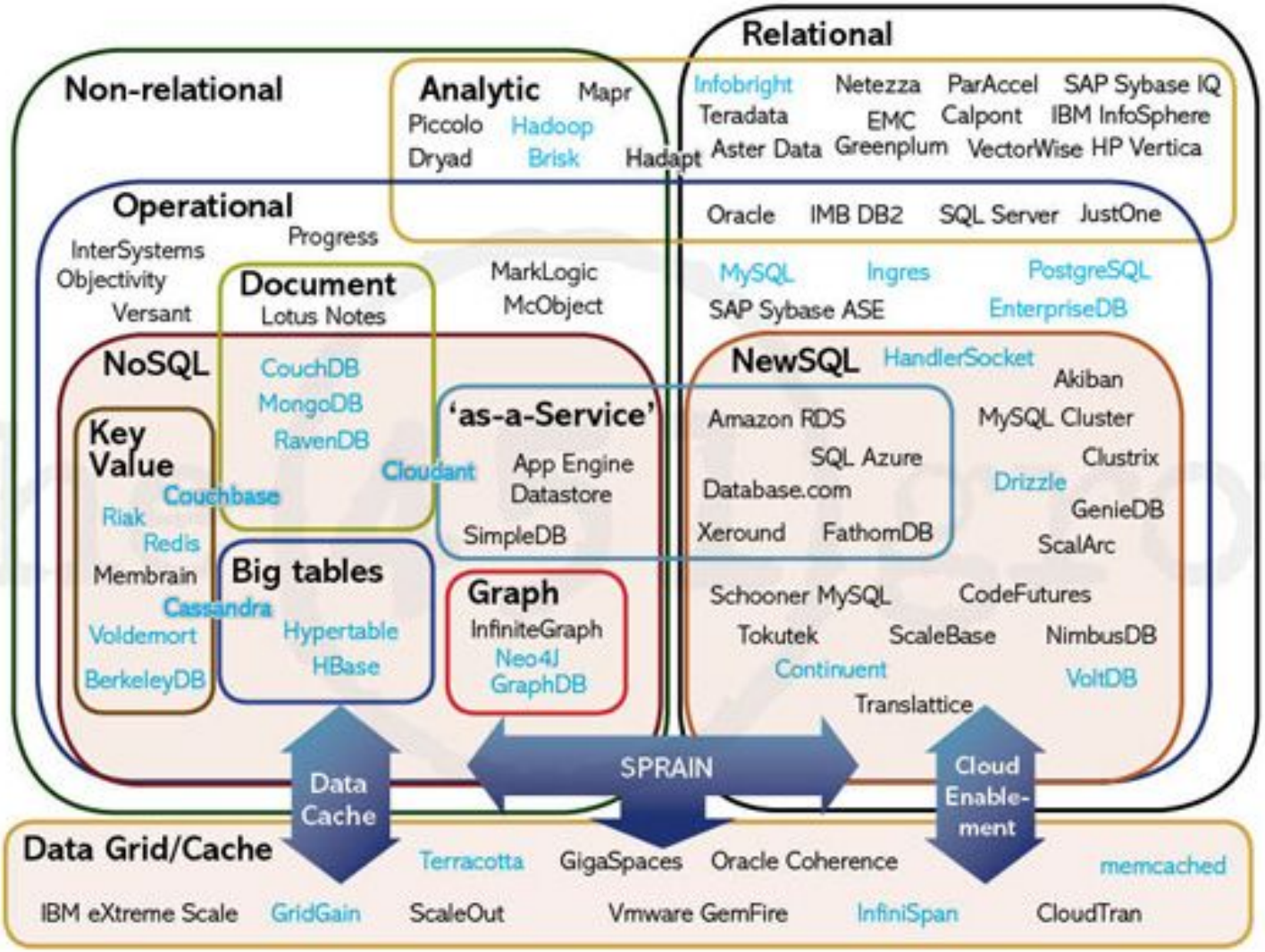
Hochschule Konstanz
Fakultät Informatik

Software Architecture

MongoDB

Prof. Dr. Marko Boger

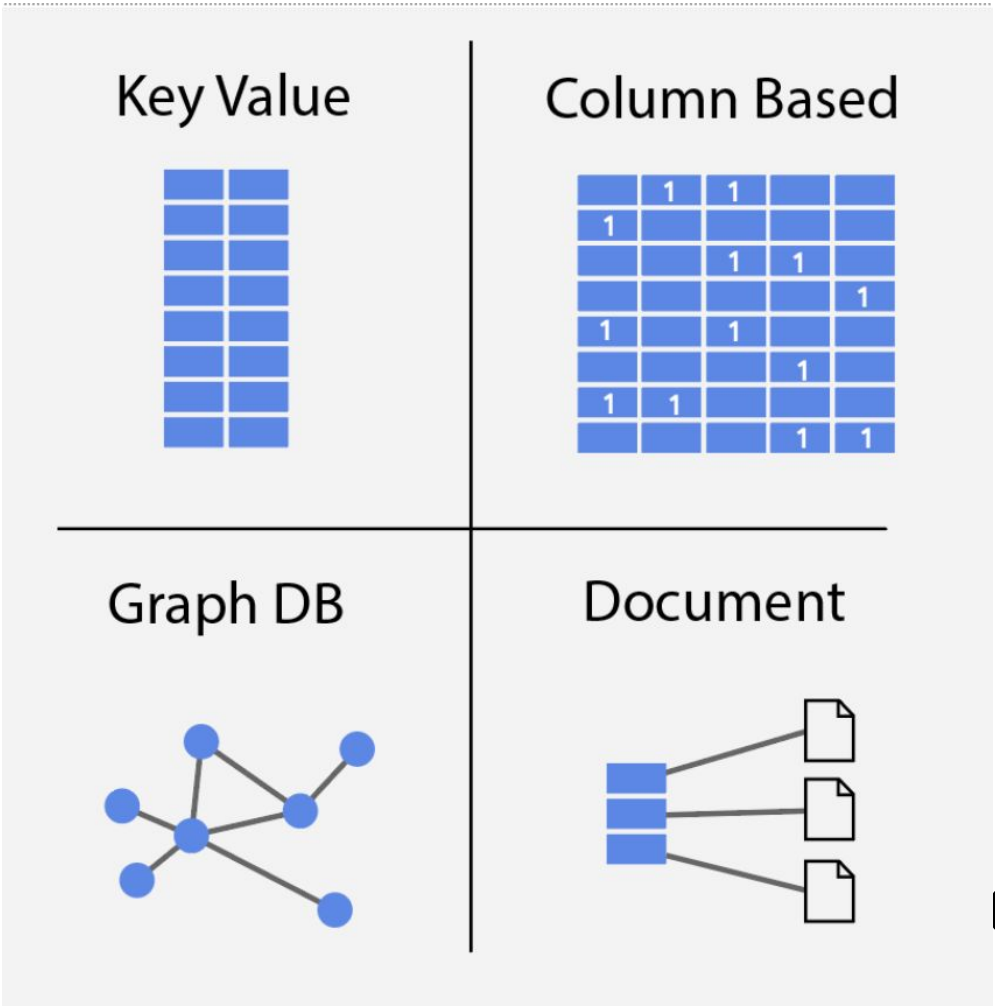
Databases


Relational
Non-relational
Analytic
Mapr
Piccolo
Hadoop
Dryad
Brisk
Hadapt
Infobright
Netezza
ParAccel
SAP Sybase IQ
Teradata
EMC
Calpont
IBM InfoSphere
Aster Data
Greenplum
VectorWise
HP Vertica
Operational
Oracle
IMB DB2
SQL Server
JustOne
InterSystems
Progress
Objectivity
Versant
Document
Lotus Notes
MarkLogic
McObject
MySQL
Ingres
PostgreSQL
SAP Sybase ASE
EnterpriseDB
NoSQL
CouchDB
MongoDB
RavenDB
Cloudant
'as-a-Service'
App Engine
Datastore
SimpleDB
NewSQL
HandlerSocket
Akiban
Amazon RDS
MySQL Cluster
SQL Azure
Clustrix
Database.com
Drizzle
GenieDB
Xeround
FathomDB
ScalArc
Key Value
Couchbase
Riak
Redis
Membrain
Cassandra
Voldemort
BerkeleyDB
Big tables
Hypertable
HBase
Graph
InfiniteGraph
Neo4J
GraphDB
Schooner MySQL
CodeFutures
Tokutek
ScaleBase
NimbusDB
Continuent
VoltDB
Translattice
Data Cache
SPRAIN
Cloud Enable-ment
Data Grid/Cache
Terracotta
GigaSpaces
Oracle Coherence
memcached
IBM eXtreme Scale
GridGain
ScaleOut
Vmware GemFire
InfiniSpan
CloudTran

NoSQL

- Describes a broad range of database technologies which are non-relational.
  - Term coined 2009/10
  - Essentially a marketing roof for „modern“ databases
  - The current interpretation of NoSQL is „Not only SQL“
- Focused towards solving specific problems which are not suited for relational storage.
  - Large data
  - High scalability
    - Usually horizontal scaling -> more hardware instead of bigger hardware
  - No fixed schema
  - Different mechanisms for queries
    - Usually no SQL

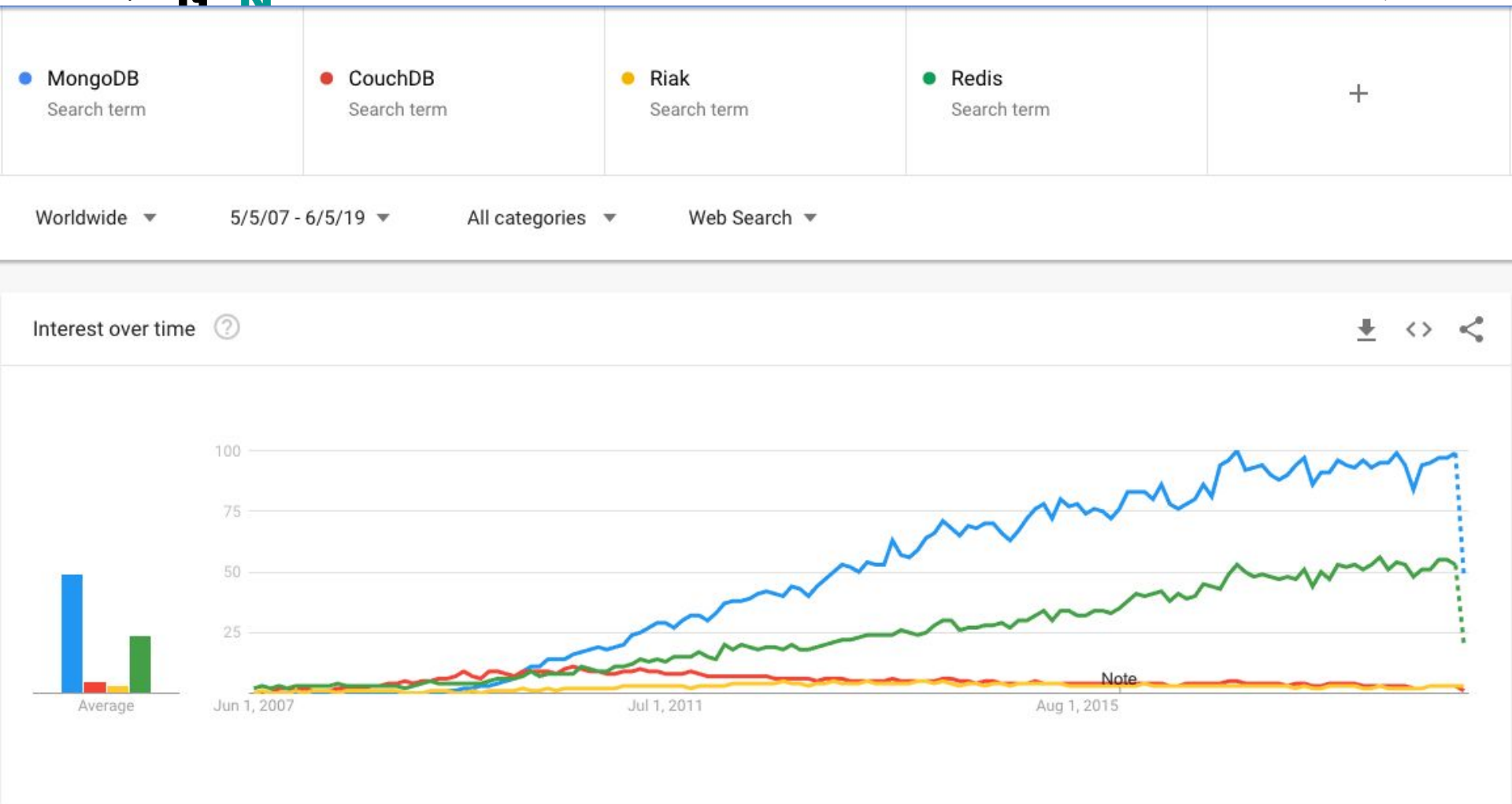
Types of NoSQL Databases



Document DBs

- Just a collection of key value pairs.
- Represented as JSON.
- No need to store things separately which naturally go together.
- Can also have attachments.
- Retrieved via key.

Document Stores compared



Scaling

Relational Databases only scale vertically to maintain ACID properties.

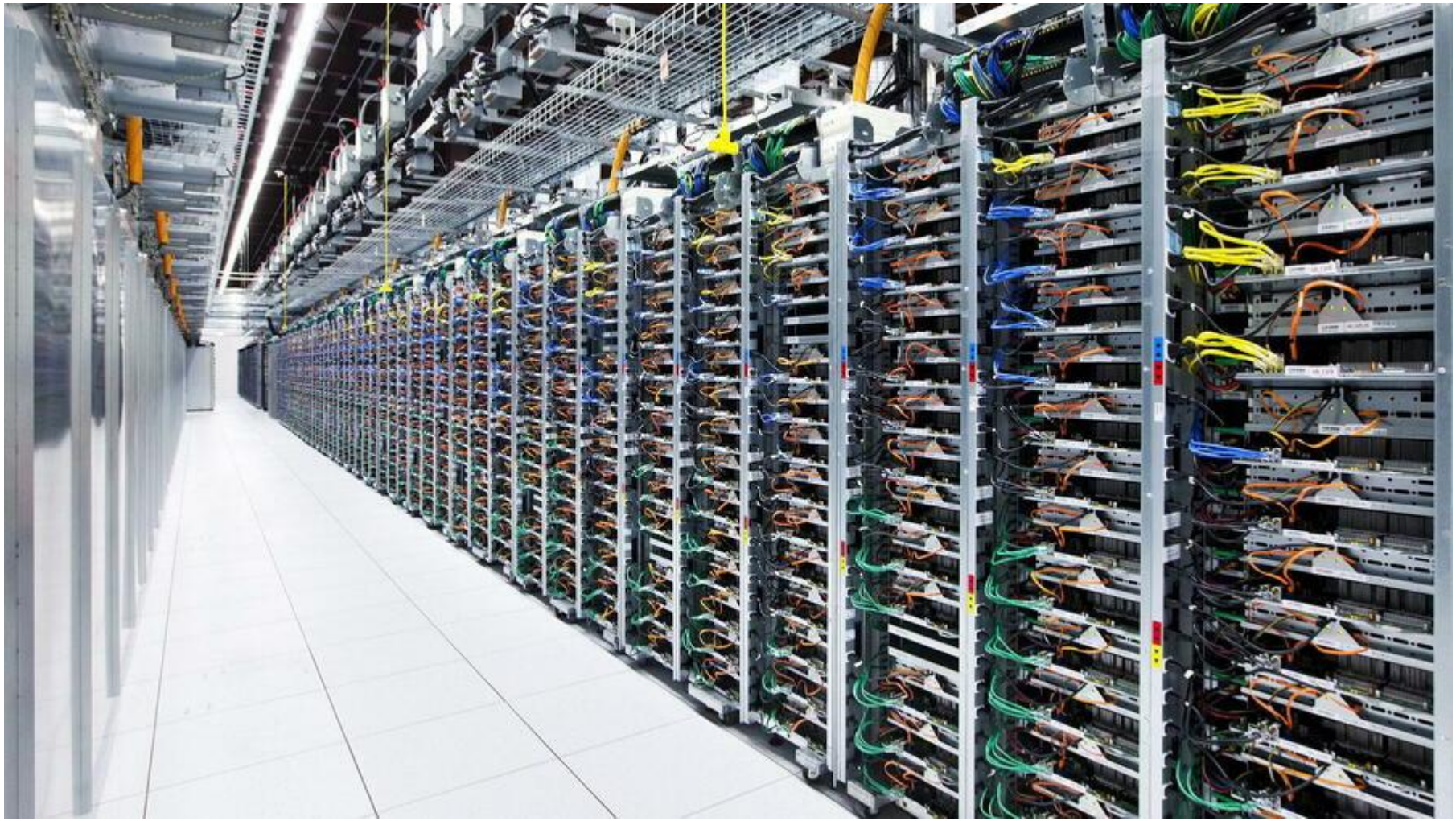
NoSQL Databases scale horizontally.

Scale Up- Vertical Scaling

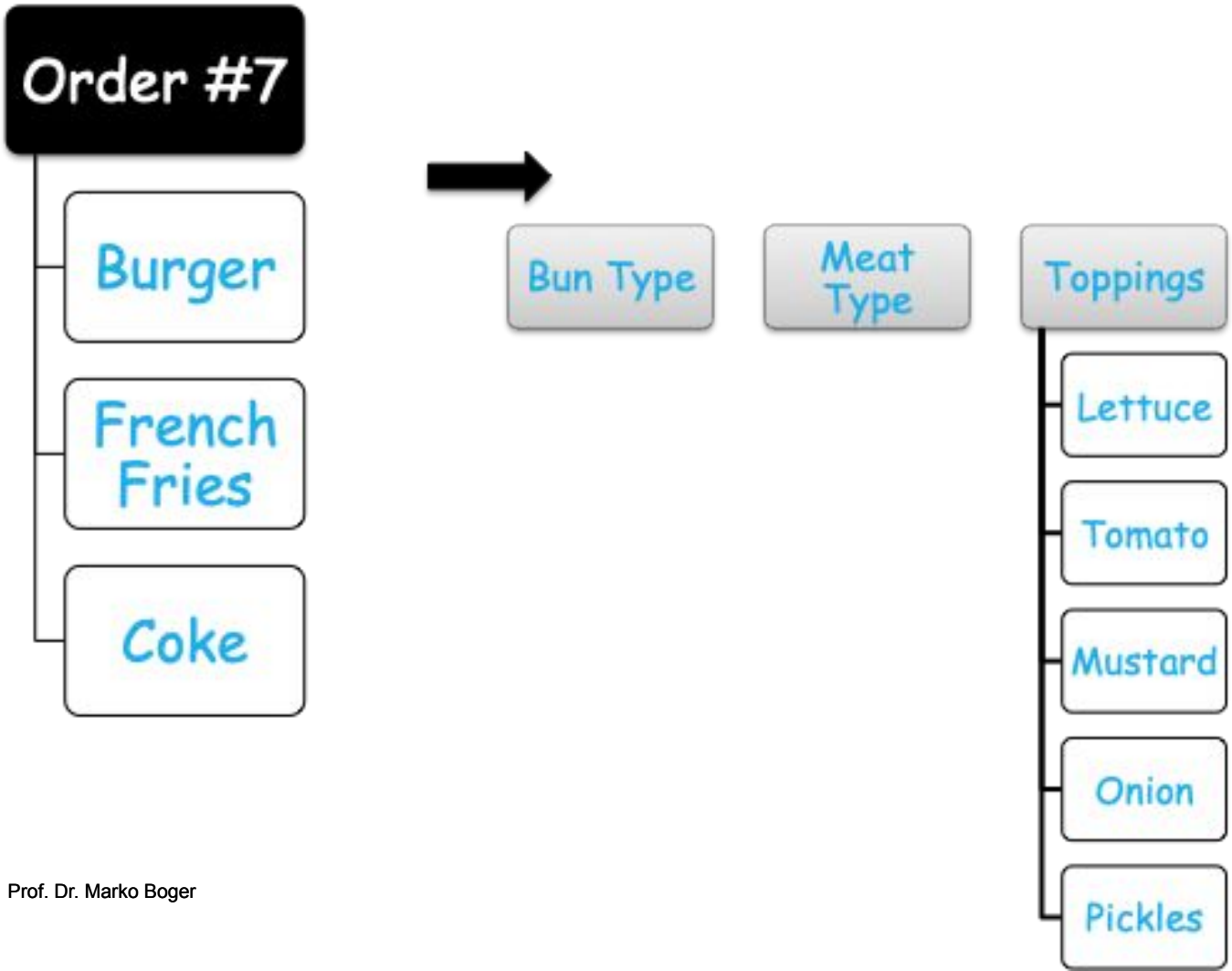
Scale Out- Horizontal Scaling

Rechenzentrum “in der Cloud”

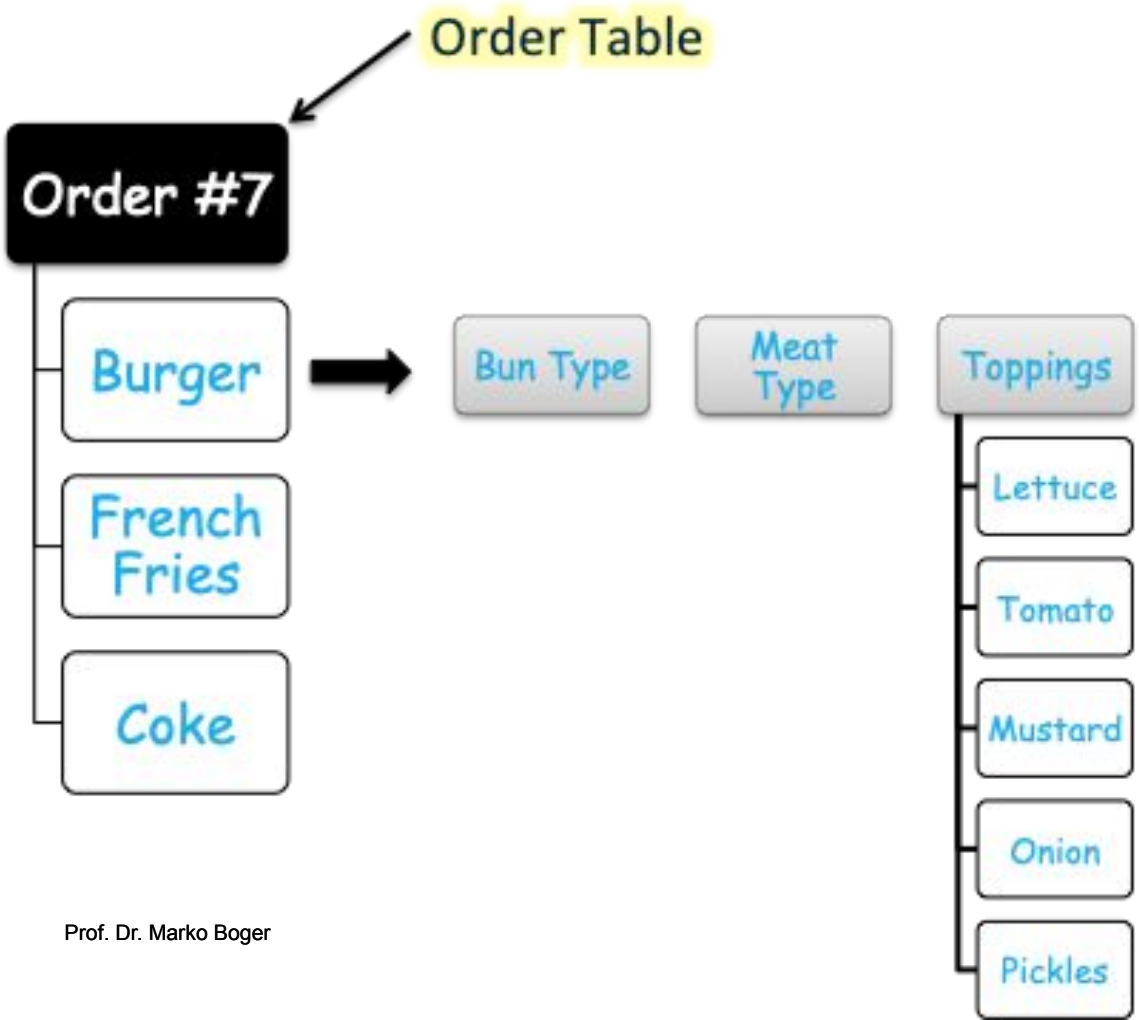
Hochschule Konstanz
Fakultät Informatik



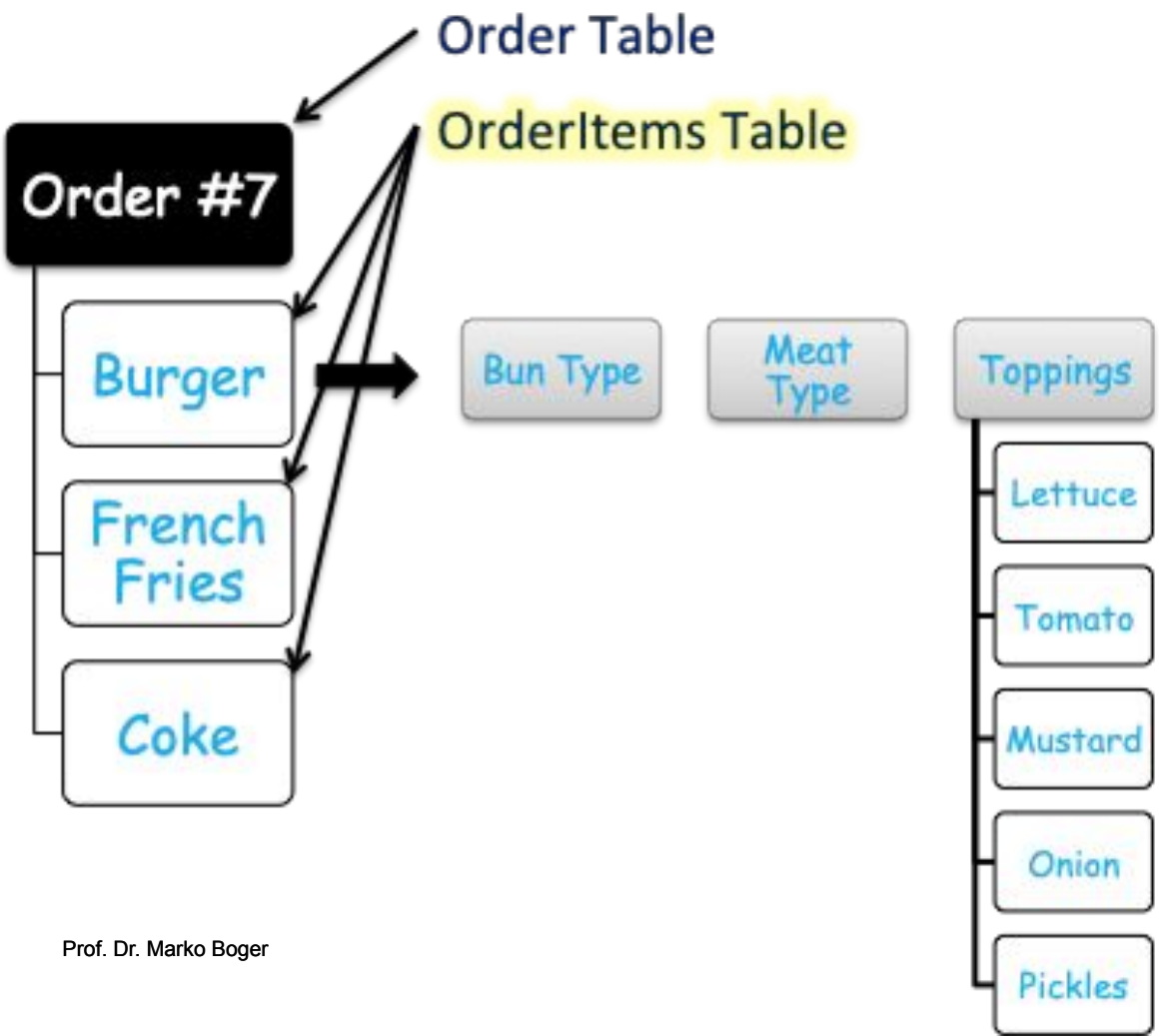
Relational Database



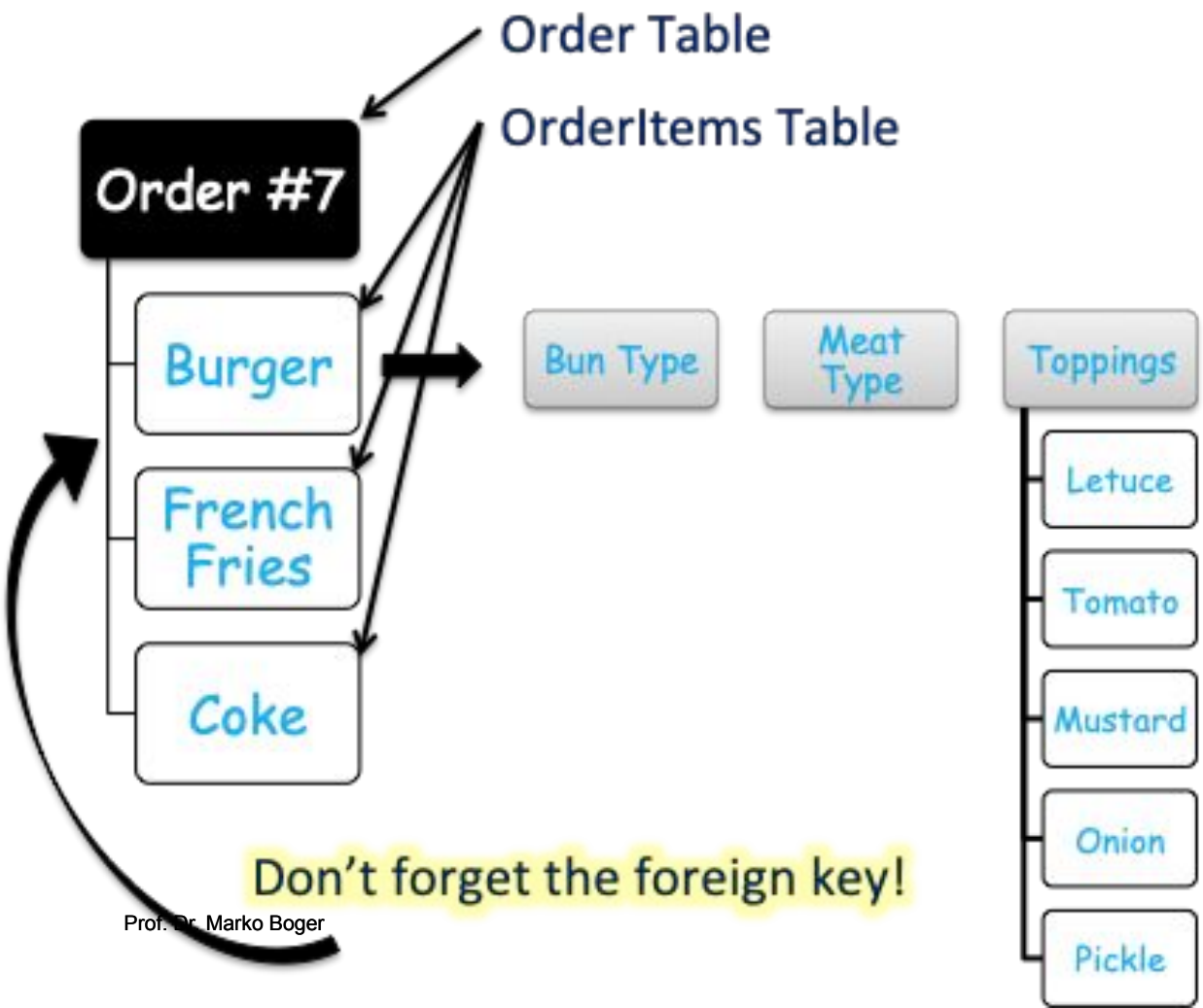
Relational Database



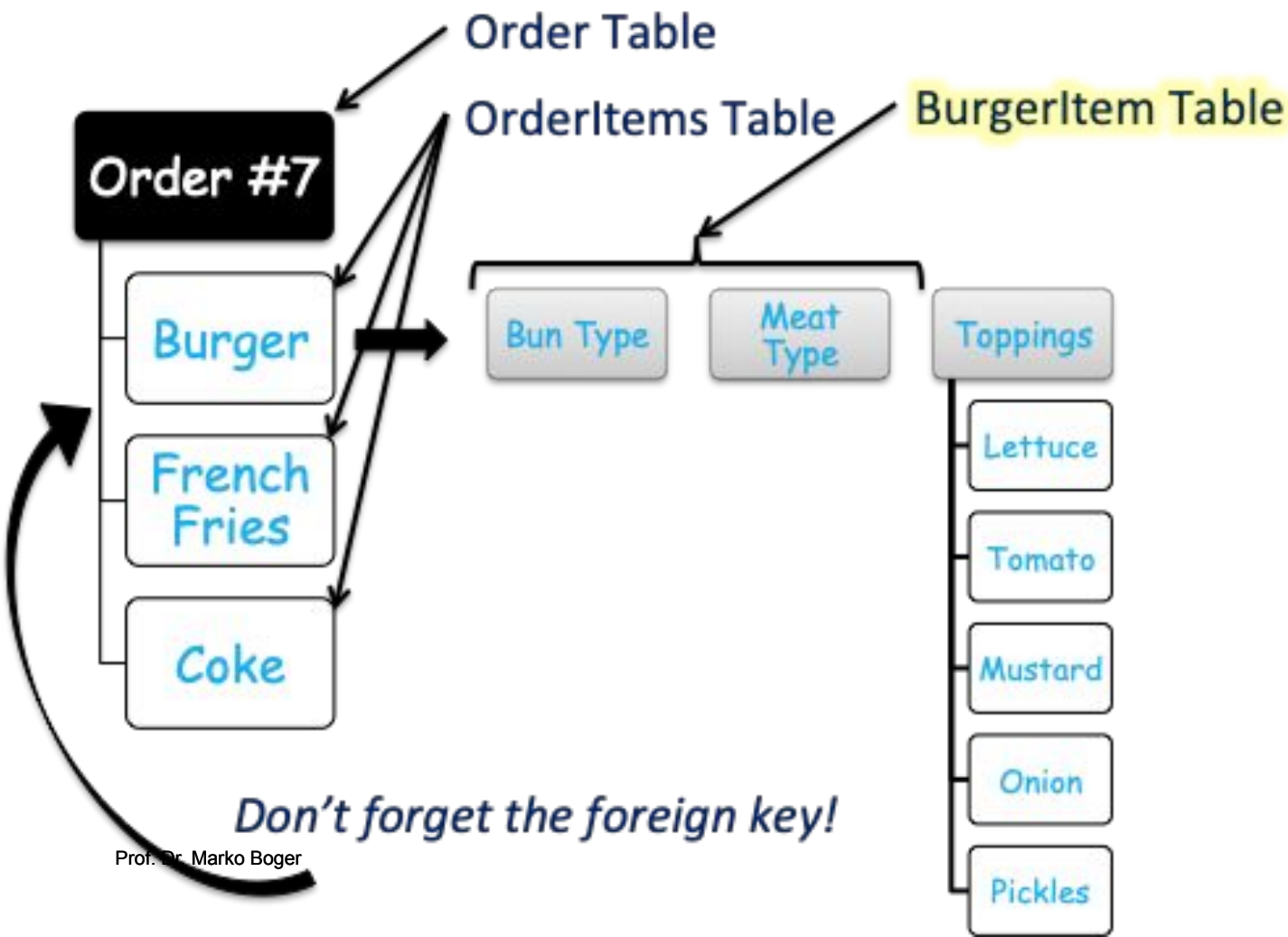
Relational Database



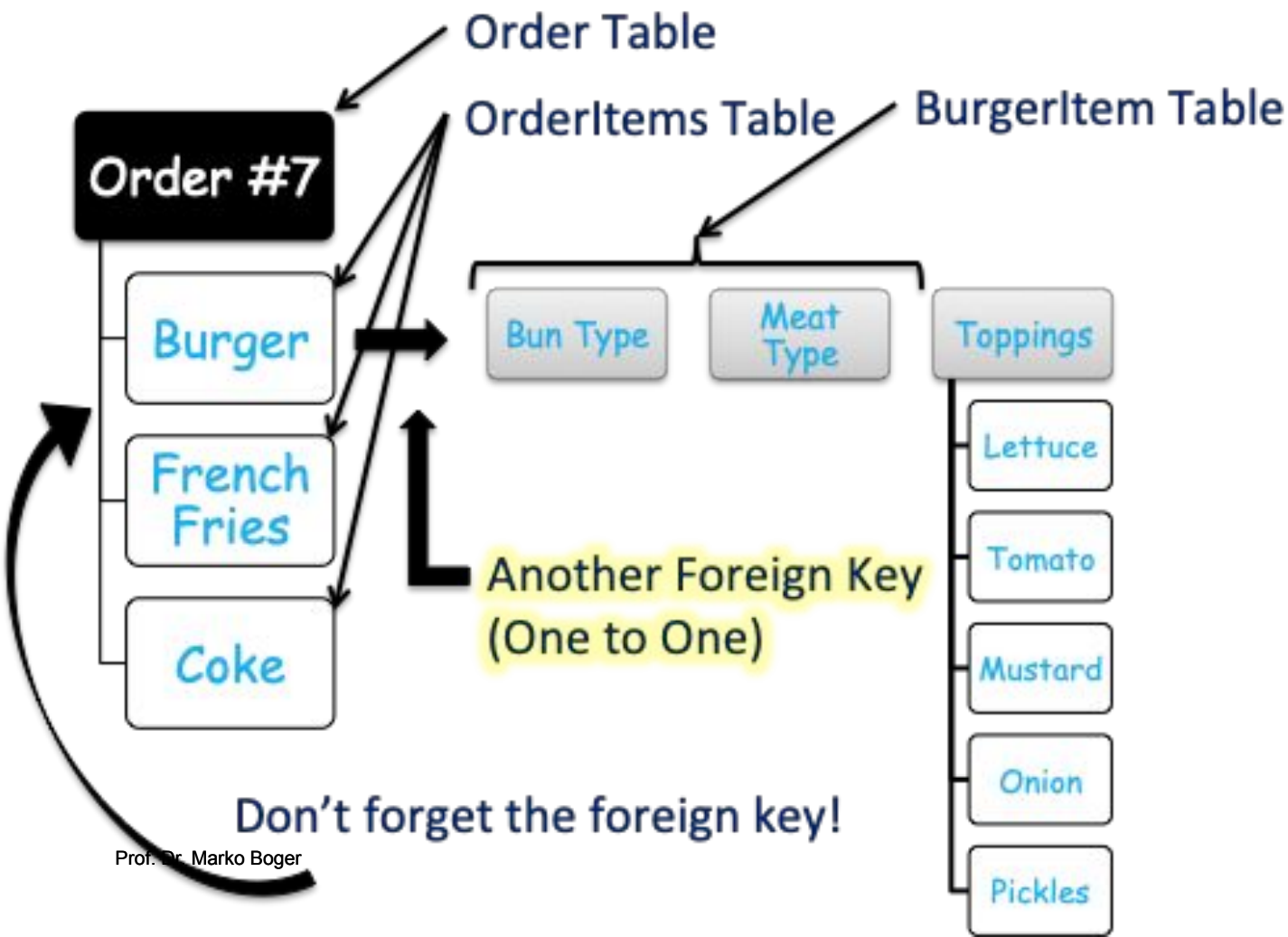
Relational Database



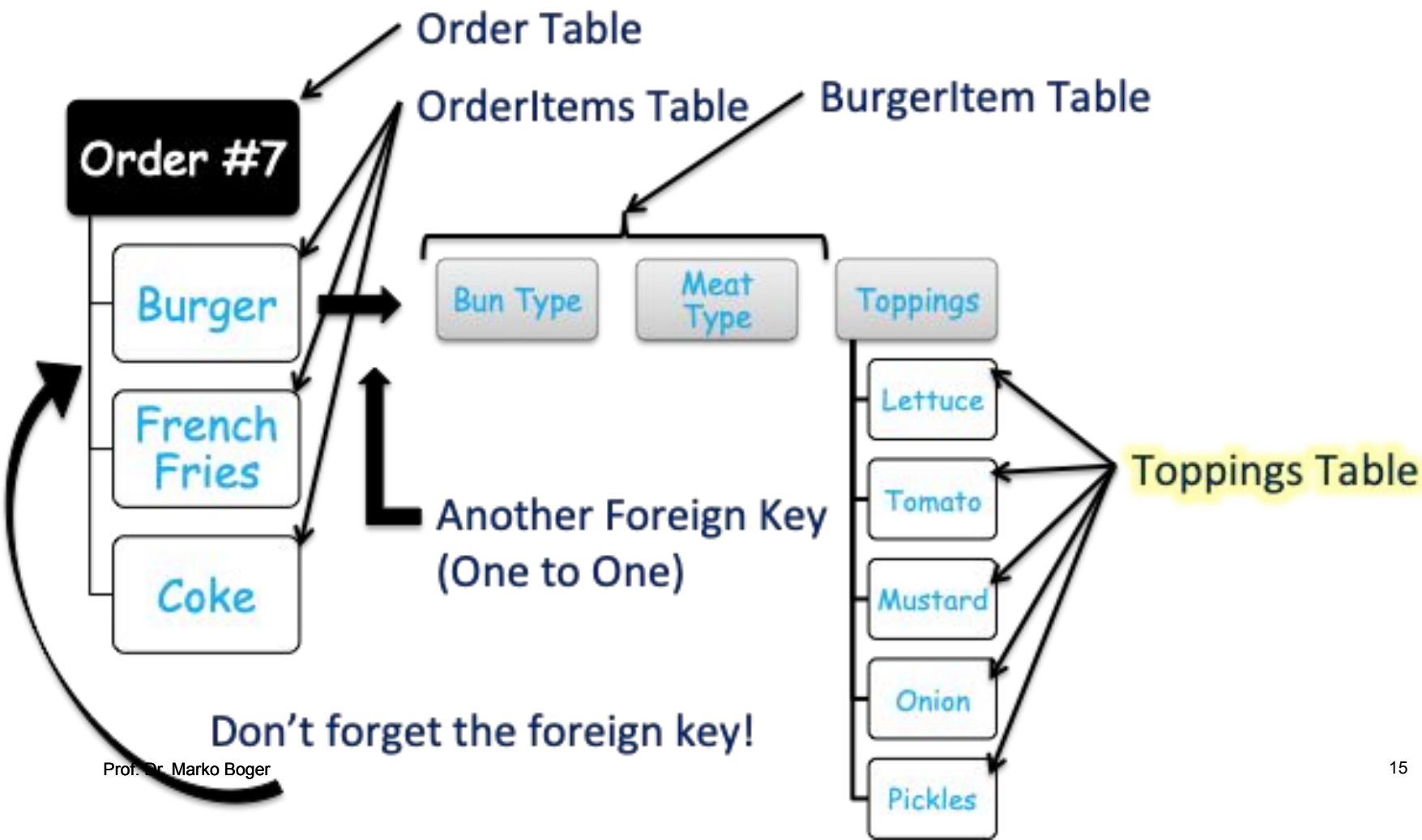
Relational Database



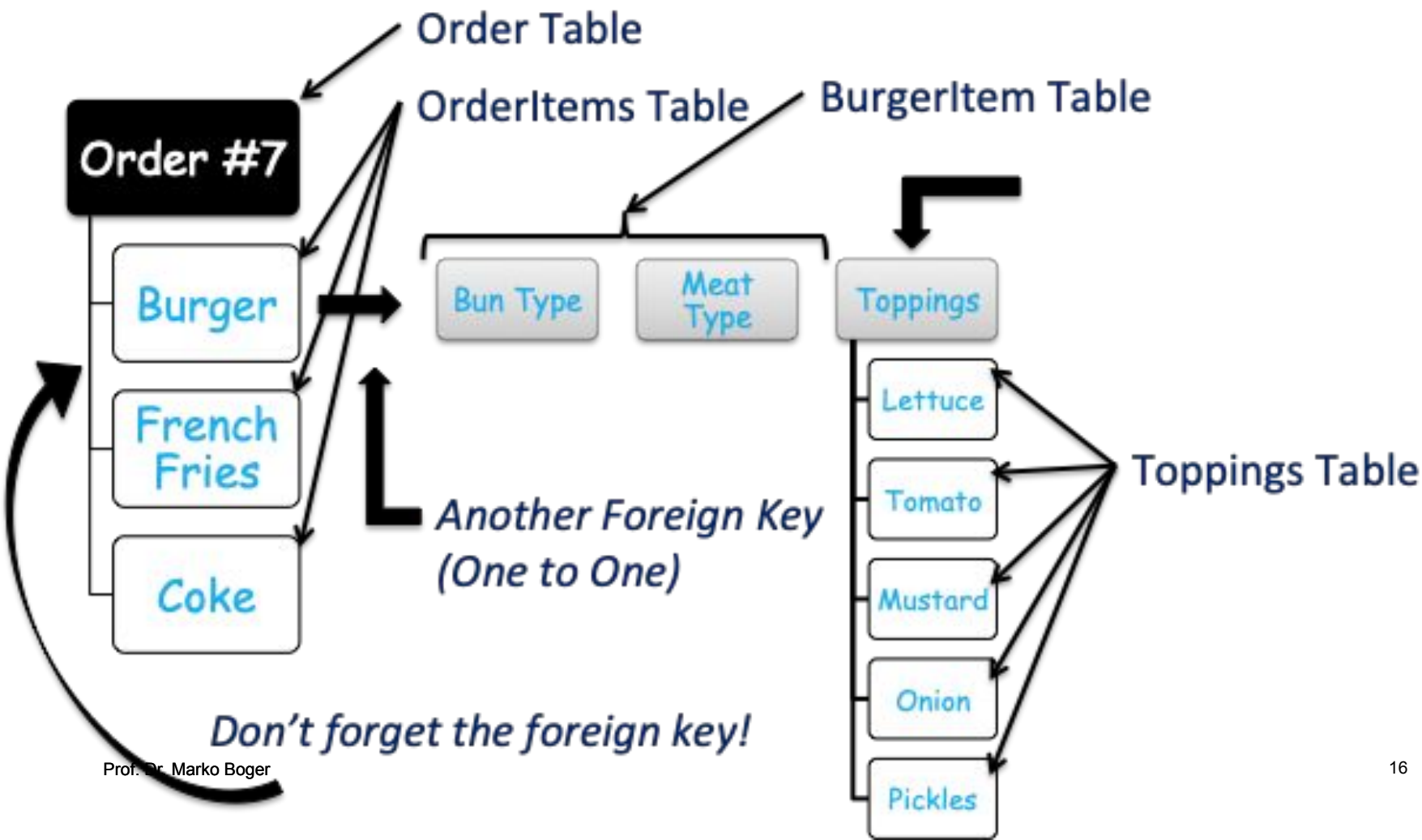
Relational Database



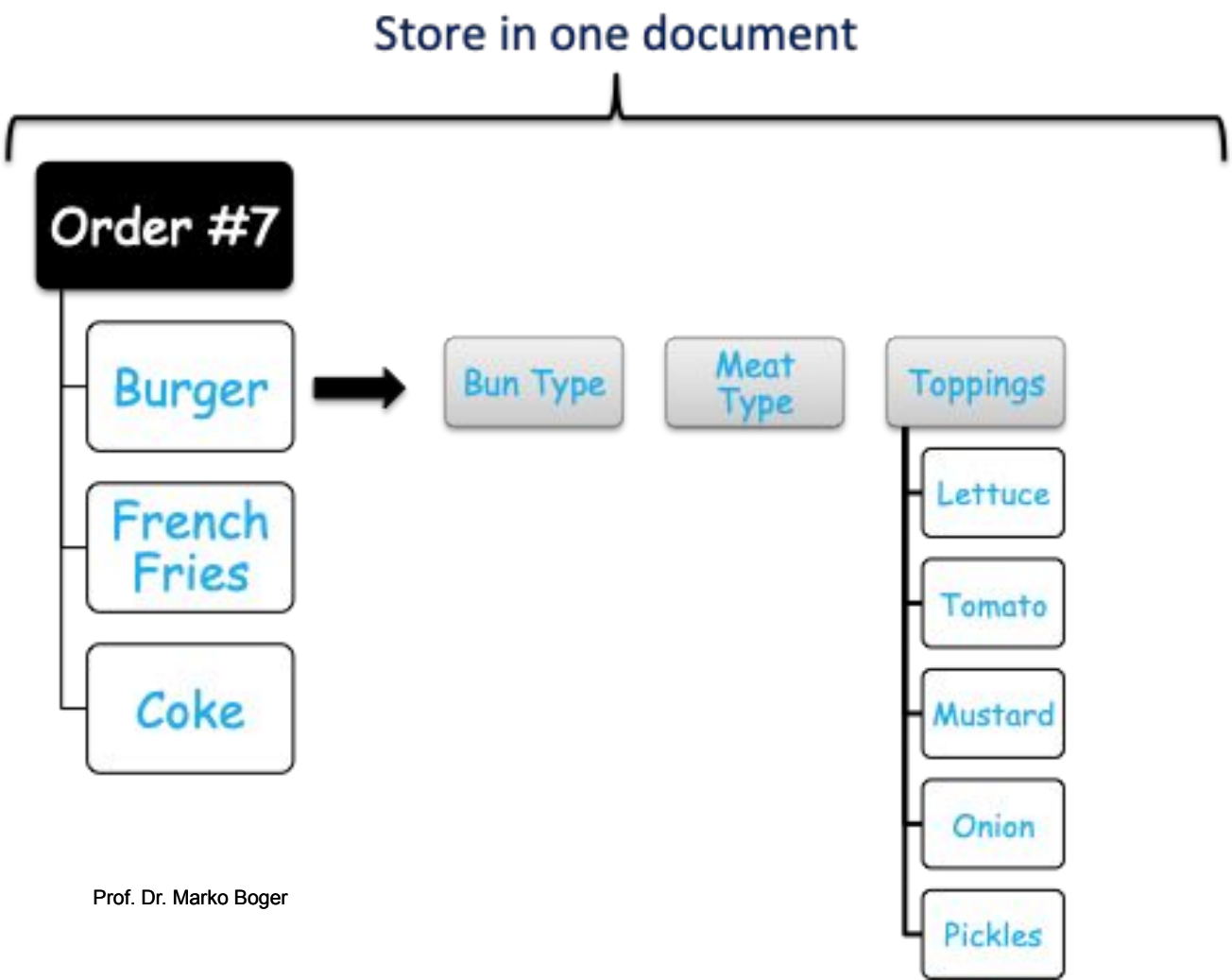
Relational Database



Relational Database



Document Database



Document Database

- Represented as JSON

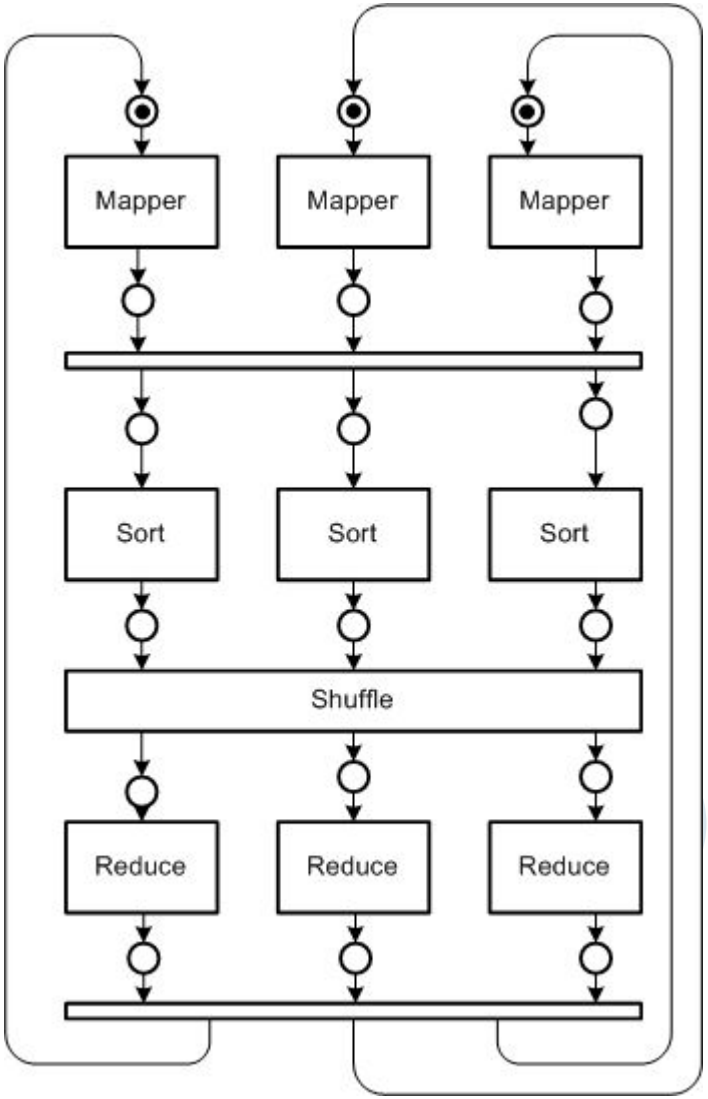
```
{
   “_id”: 7,
   “Items”: [
      {
         “Type”: “Burger”,
         “BunType”: “Sesame Seed”,
         “MeatType”: “1/4 lb”,
         “Toppings”: [“Lettuce”,”Tomato”,”Mustard”,”Onion”,”Pickles”]
      },
      {“Type”: “French Fries”},
      {“Type”: “Coke”}
   ]
}
```

Map/Reduce

Google's Framework for distributed data processing

Idea:

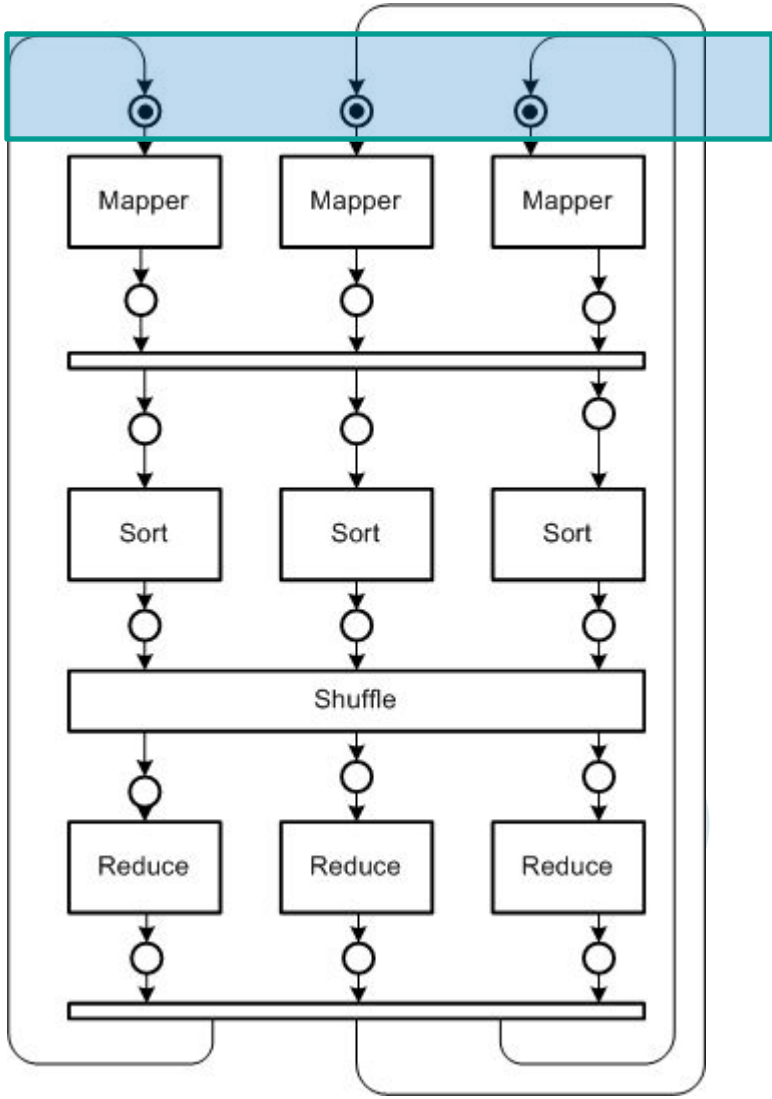
- Shared-Nothing
- Data processing distributed over worker nodes
- Mapping defines dependencies between data records
- Reducer defines aggregation operation



Map/Reduce – Blog Analysis

```
function map(document) {
        emit(document.author, 1);
}
```

```
function reduce(key, values) {
        emit(key, values.length);
}
```



Map/Reduce – Blog Analysis

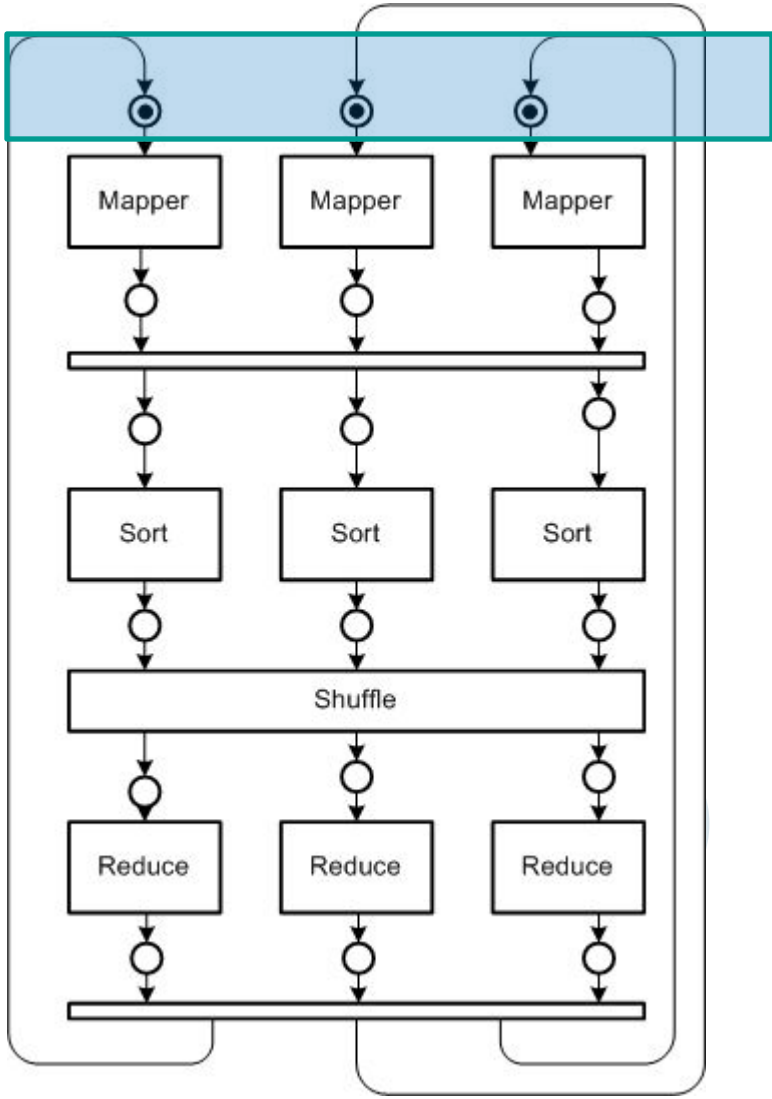
Task: Find number of blog-entries per author

```
function map(document) {
        emit(document.author, 1);
}
```

```
function reduce(key, values) {
        emit(key, values.length);
}
```

Input data:

{ „id“ = „123“, „text“ = „…“, author = „john doe“ }
{ „id“ = „124“, „text“ = „…“, author = „jane doe“ }
{ „id“ = „125“, „text“ = „…“, author = „jane doe“ }
{ „id“ = „126“, „text“ = „…“, author = „john doe“ }
{ „id“ = „127“, „text“ = „…“, author = „max doe“ }



Map/Reduce – Blog Analysis

Task: Find number of blog-entries per author

```
function map(document) {
        emit(document.author, 1);
}
```

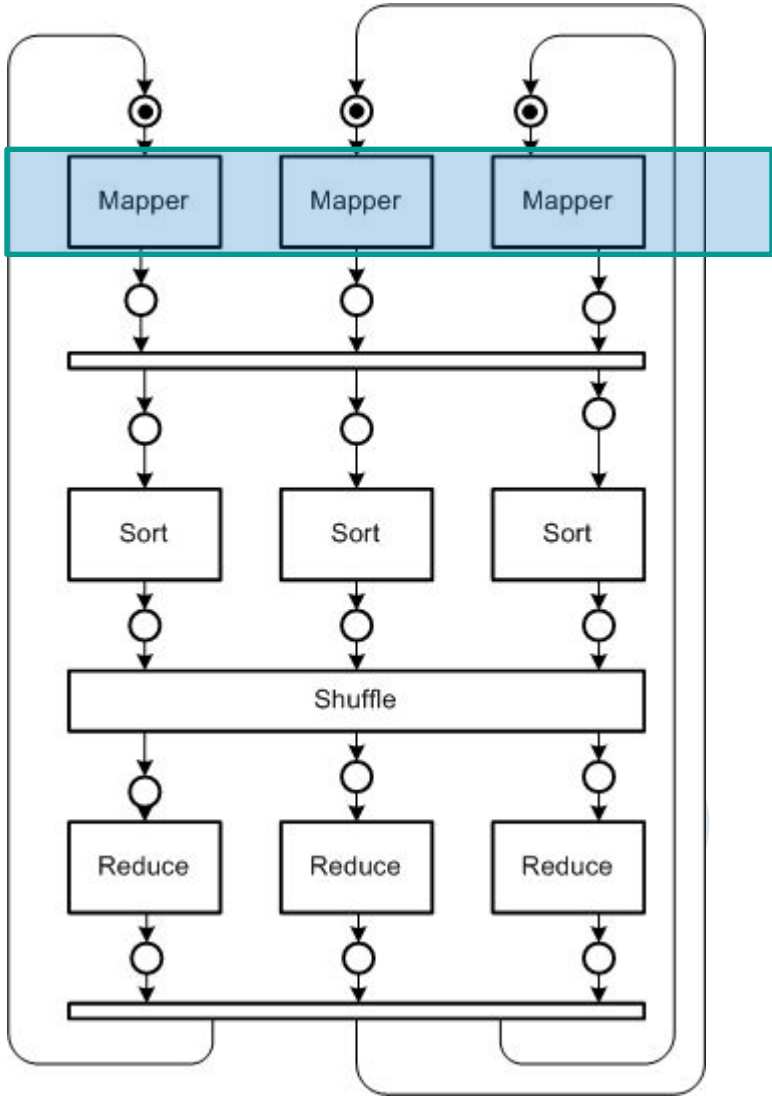
```
function reduce(key, values) {
        emit(key, values.length);
}
```

Input data:

{
{
{
{

Mapped data:

„Key“ = „john doe“ , „Value“ = „1“
„Key“ = „jane doe“ , „Value“ = „1“
„Key“ = „jane doe“ , „Value“ = „1“
„Key“ = „john doe“ , „Value“ = „1“
„Key“ = „max doe“ , „Value“ = „1“



Map/Reduce – Blog Analysis

Task: Find number of blog-entries per author

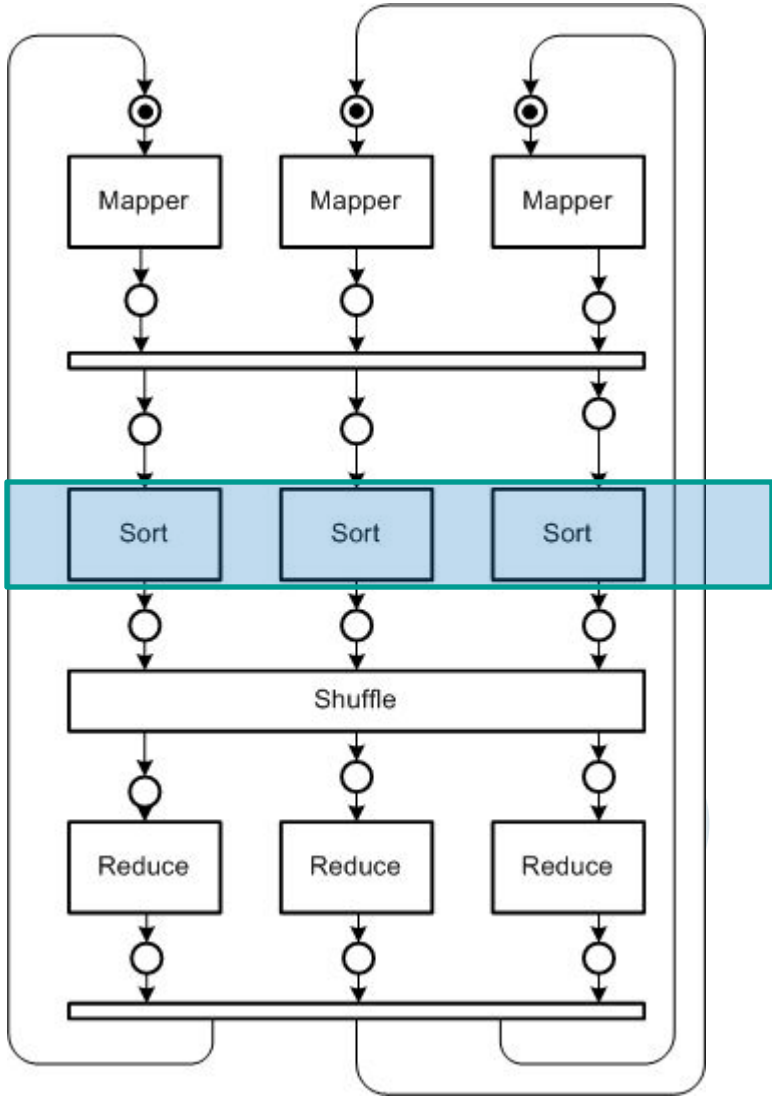
```
function map(document) {
        emit(document.author, 1);
}
```

```
function reduce(key, values) {
        emit(key, values.length);
}
```

Input data:

Sorted data:

„Key" = „jane doe", „Value" = „1"
„Key" = „jane doe", „Value" = „1"
„Key" = „john doe", „Value" = „1"
„Key" = „john doe", „Value" = „1"
„Key" = „max doe", „Value" = „1"

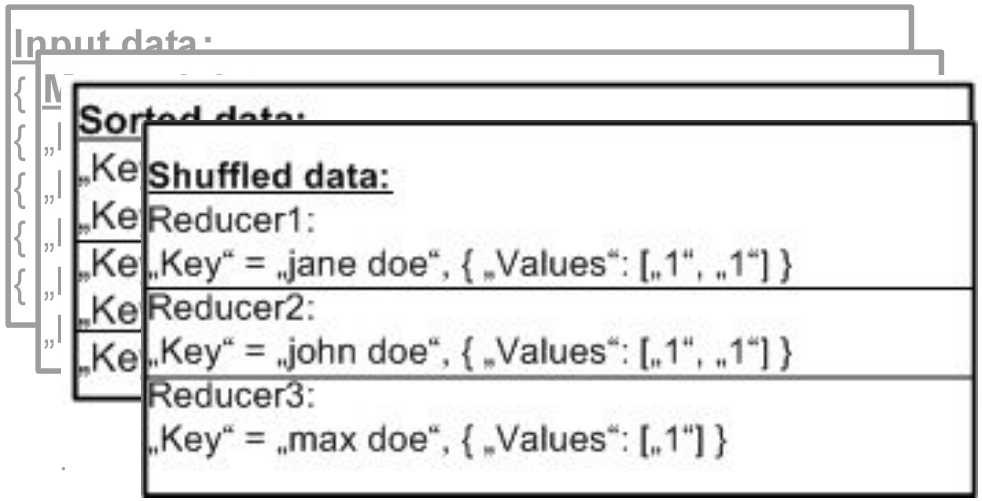


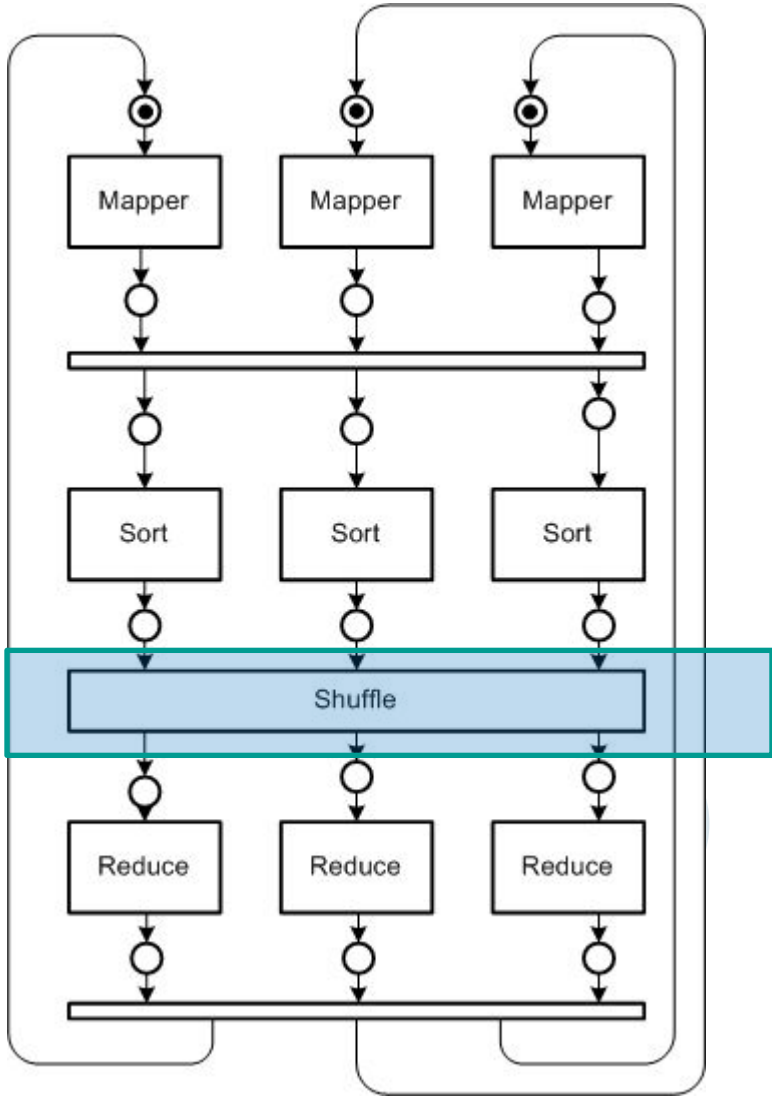
Map/Reduce – Blog Analysis

Task: Find number of blog-entries per author

```
function map(document) {
        emit(document.author, 1);
}
```

```
function reduce(key, values) {
        emit(key, values.length);
}
```



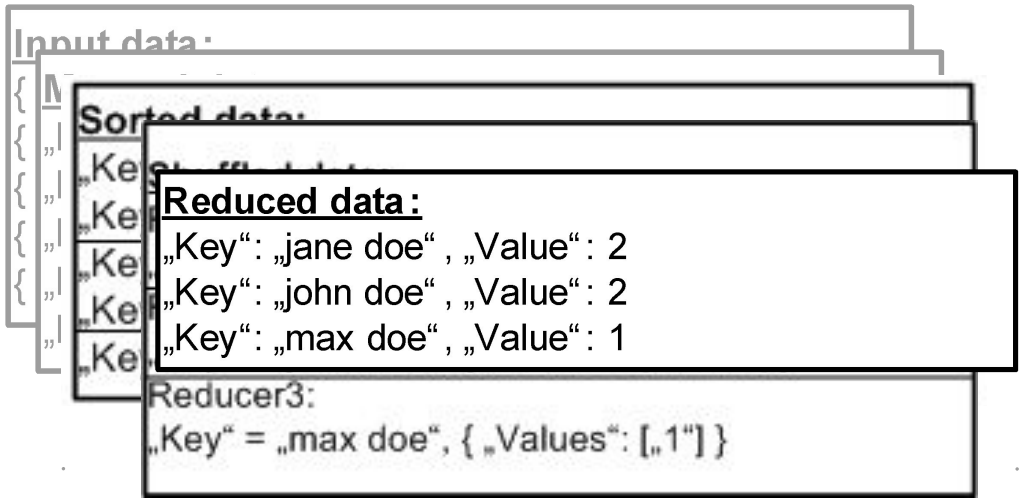


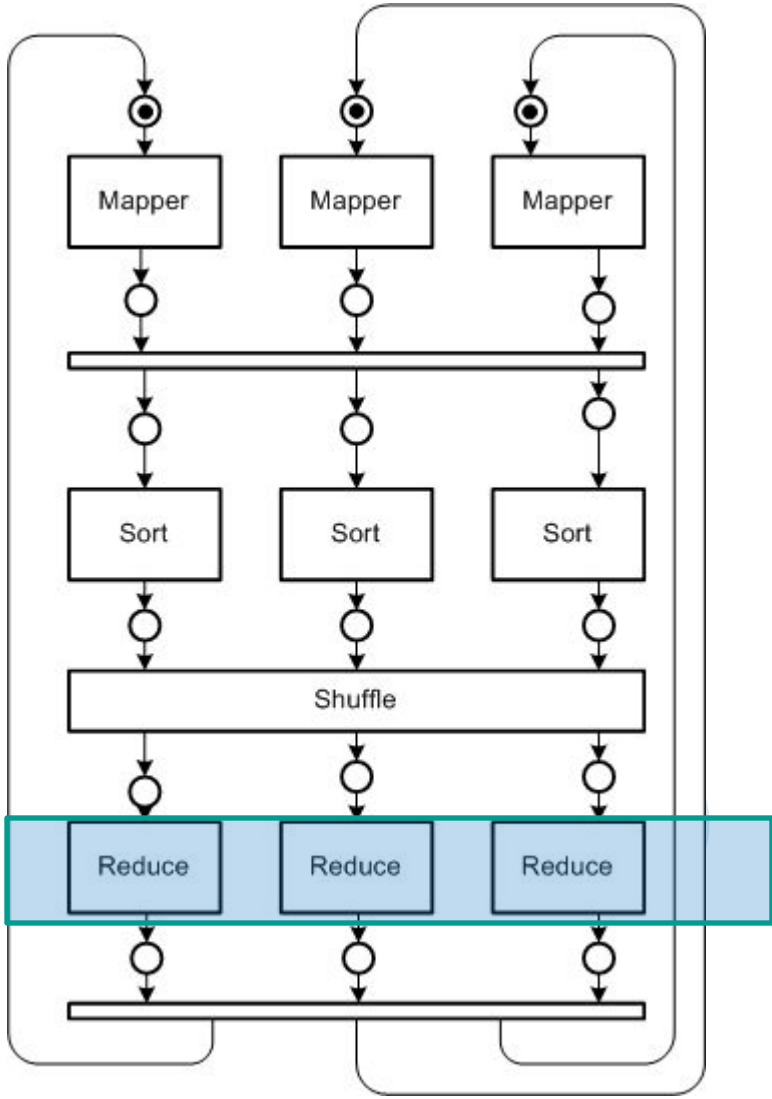
Map/Reduce – Blog Analysis

Task: Find number of blog-entries per author

```
function map(document) {
        emit(document.author, 1);
}
```

```
function reduce(key, values) {
        emit(key, values.length);
}
```





Relational Tables vs JSON Documents

MongoDB in 5 Minutes with Eliot Horowitz, CTO of MongoDB

https://www.youtube.com/watch?v=EE8ZTQxa0AM

Installing MongoDB

Install MongoDB Community Edition 4.0 according to your platform

```
brew tap mongodb/brew
```

```
brew install mongodb-community@4.0
```

Start Mongo Daemon

```
mongod --config /usr/local/etc/mongod.conf
```

Connect to the Database

```
mongo
```

Load MongoDB Scala Driver in sbt

```
libraryDependencies += "org.mongodb.scala" %% "mongo-scala-driver" % "2.6.0"
```

Using mongo

Mongo databases can be maintained and accessed from the console

use <somedb>

db.users.insert(<document>)

db.users.find()

MongoDB Structure

A MongoDB can have several Databases. `use <database>` switches to that database or creates it.

Within a database it stores collections.

`db.<collection>.insert(<document>)` creates a collection and/or adds documents to it.

db.<collection>.find() retrieves all documents

db.<collection>.find(<query>) retrieves only matching documents

Further commands in a Cheat Sheet

Mongo Scala Driver

Mongo's API in Scala is centered around a MongoCollection case class. Obtain that.

```
val client: MongoClient = MongoClient()
val database: MongoDatabase = client.getDatabase("mydb")
val collection: MongoCollection[Document] = database.getCollection("mycoll")
```

Then work with this object

```
val document: Document = Document("_id" -> 1, "x" -> 1)
collection.insertOne(document)
```

This returns an observable object

```
val insertObservable: Observable[Completed] = collection.insertOne(document)

insertObservable.subscribe(new Observer[Completed] {
  override def onNext(result: Completed): Unit = println(s"onNext: $result")
  override def onError(e: Throwable): Unit = println(s"onError: $e")
  override def onComplete(): Unit = println("onComplete")
})
```

Task 7

Use MongoDB to build a second DB implementation.

Again, use the DAO Pattern to make the interface independent of the used DB.